DrainSinkhorn: Safe Elimination for Batched Optimal Transport

Abstract

Flash-class kernels make one Sinkhorn update fast. Batched workloads expose a second optimization target: stop executing problems verified as converged. DrainSinkhorn realizes this safe elimination. After each alternating scaling cycle, it uses the updated-marginal structure of Sinkhorn to screen a batch cheaply, invokes the original two-sided residual verifier only on screen-positive lanes, and physically removes verified lanes from every candidate-indexed tensor. The numerical objective and release criterion are unchanged; subsequent kernels simply become narrower.

A two-host Packer19 study isolates the execution mechanisms. Batched residual evaluation contributes 1.073×, the Sinkhorn-specific one-sided screen adds 1.055×, and a matched logical-mask control shows that physical draining adds 1.224× while reducing physical candidate-update slots from 256 to 156. Each effect wins 24/24 paired timing units. Shadow replay finds no already-releasable lane among 1488 screen-negative events. On a real MetroPT-3 predictive-maintenance endpoint, draining provides 1.745× over matched static execution and reduces endpoint energy by 1.783×. Complete deployments reach 4.072× on MetroPT-3 and 14.193× on Packer19; a separate real ImageNet-32 feature-space OT-Flow-Matching pipeline obtains 2.786× while passing its frozen NFE 4/8/16 quality checks. A fixed-work intervention shows that draining gains increase when the same problems are grouped into more heterogeneous stopping depths. These results establish safe elimination as a workload-level optimization that composes with fast inner Sinkhorn kernels.

1 Introduction

Recent progress in entropic optimal transport (EOT) has concentrated on making one Sinkhorn update cheaper. Tiled, matrix-free, and fused kernels reduce memory traffic and expose a single solve to the GPU [4, 5, 15]. Modern workloads, however, rarely stop at one solve. Industrial pipelines, scientific state transitions, and minibatch coupling builders repeatedly invoke Sinkhorn on many complete problems. Once the inner kernel is fast, their next bottleneck is no longer only the cost of an update; it is how many candidate updates the workload continues to execute.

If candidates require different numbers of iterations, a fixed-width batch follows its slowest member. Finished candidates continue to occupy state, audit bandwidth, and kernel lanes. The formal Packer19 path in table 1 shows why execution control matters: width-one execution beats the corresponding static packed path (ratio 0.907×). A useful abstraction must decide both *when* a numerical problem has finished and *whether* removing it repays the control cost.

We introduce DrainSinkhorn, safe elimination for batched Sinkhorn solves. The name describes the execution: Sinkhorn balances transport mass, while DrainSinkhorn drains verified-finished *problem instances* from the batch. It never drains mass. The method couples three operations that must agree on the same numerical state. First, alternating scaling makes one marginal current by construction and enables a cheap batched screen of the other marginal. Second, screen-positive lanes are replayed through the registered two-sided residual verifier; the screen never authorizes release. Third, a verified lane is removed from the potentials, marginals, supports, and scratch buffers, so the next kernel executes at a smaller physical width.

This mechanism is distinct from merely placing independent loops in a batch. General control-flow batching, iteration-level serving, and batched numerical solvers already address divergent iteration counts [7, 10, 14, 16]. DrainSinkhorn contributes the numerical control plane required for Sinkhorn: an operator-specific screen, the original two-marginal release contract, and physical compaction of EOT state. FlashSinkhorn and related systems lower the cost of each inner update; DrainSinkhorn lowers the candidate-update work executed across a workload. The two optimizations compose.

The original motivation came from positive fixed-point iterations such as PageRank: convergence structure can expose work ready to leave the critical path. In Sinkhorn, nonlinear Perron–Frobenius (PF) geometry provides a local mechanism for strict-tail and first-passage-depth variation. Sinkhorn algebra provides the marginal identity, the current two-sided residual certifies release, and physical compaction realizes the speedup.

Our contributions are:

1. **Safe elimination for Sinkhorn.** We give a screen–verify–drain execution contract that preserves the EOT objective and registered release criterion while shrinking subsequent candidate-indexed kernels.
2. **A work law for heterogeneous depths.** We connect the observed release-depth survival curve to executed candidate slots and a measured width-cost curve, separating an exact work identity from a hardware timing

model.

3. **Closed component attribution.** Two matched Packer19 campaigns isolate batched audit, one-sided screening, logical masking, physical draining, backend differences, and the boundary of profitable packing. The decisive control keeps verified lanes logically frozen at physical width eight; only physical removal reduces subsequent GPU work.
4. **Real endpoint and scale evidence.** MetroPT-3, Packer19, real ImageNet-32 feature-space OT-FM, and complete-device multi-GPU experiments carry the same execution contract from component effects to deployed wall time, energy, and training.

The resulting message is simple: a faster update does not eliminate a finished solve. DrainSinkhorn turns verified numerical completion into work that the GPU no longer performs.

2 Setting and Prior Systems

Entropic optimal transport. For positive unit-mass marginals $a \in \mathbb{R}^n_+$ and $b \in \mathbb{R}^m_+$, EOT solves

$$\min_{\pi \geq 0} \langle C, \pi \rangle + \varepsilon \sum_{ij} \pi_{ij}(\log \pi_{ij} - 1), \quad \pi \mathbf{1} = a, \quad \pi^\top \mathbf{1} = b. \tag{1}$$

With $K = \exp(-C/\varepsilon)$, Sinkhorn alternates diagonal scalings to form $\pi = \operatorname{diag}(u) K \operatorname{diag}(v)$ [3]. Coordinate, relaxed, low-rank, and Newton methods reduce arithmetic or iteration count inside one solve [1, 8, 11, 12]. FlashSinkhorn instead rewrites stabilized updates as streaming LogSumExp reductions and lowers their IO cost [15]. DrainSinkhorn is an outer execution layer: it keeps the selected inner update and changes which verified-unfinished problems receive the next one.

Initialization and repeated couplings. Carry, analytic extensions, and learned dual proposals can reduce a new problem's starting defect [2, 13]. Large-coupling Flow Matching also uses soft c-transforms when supports change [17]. These proposals affect release depth but are optional: the formal Packer19 attribution uses the same frozen proposal in every arm, and the current two-sided residual remains the authority.

Dynamic batching and batched solvers. Automatic batching can transform data-dependent programs by tracking the program point of each member [10]. Orca schedules autoregressive inference one iteration at a time and selectively batches compatible operations [16]. Ginkgo fuses batches of uniform independent linear systems and records per-item convergence [7]. jaxipm redesigns IPOPT around heterogeneous iteration fusion and iteration-level batching, replacing completed optimization problems to sustain throughput [14]. These systems establish the general value of handling variable iteration counts. DrainSinkhorn specializes that principle to the numerical structure of EOT: the Sinkhorn marginal screen, two-sided release semantics, and compaction of candidate-indexed OT state form one algorithmic contract.

OT consumers. POT and OTT provide broad transport APIs [4, 6]. Multisample Flow Matching consumes an OT coupling to select training pairs [9]; the couplings can be prepared independently of network optimization [17]. We evaluate the complete coupling-to-consumer boundary for industrial endpoints and carry the same execution path through a fixed-update feature-space OT-FM training pipeline.

3 DrainSinkhorn

3.1 The marginal identity behind the screen

For one candidate, a Sinkhorn cycle performs

$$u^{k+1} = a \oslash (K v^k), \qquad v^{k+1} = b \oslash (K^\top u^{k+1}), \tag{2}$$

where $\oslash$ denotes elementwise division. Immediately after the second leg, exact arithmetic gives

$$v^{k+1} \odot (K^\top u^{k+1}) = b. \tag{3}$$

The column marginal is therefore current by construction; the row marginal $u^{k+1} \odot (K v^{k+1})$ is the side that can remain outside tolerance. The implementation evaluates the row residual for all active lanes in one batched operation. Alternating scaling supplies this identity; PF geometry enters the separate account of release-depth variation in section 4.

3.2 Screen, verify, and drain

Let $\widehat{r}^{\mathrm{row}}_{t,k}$ be the batched screen value. A lane whose screen exceeds the configured threshold continues without invoking the more expensive plan-application verifier. A screen-positive lane is recomputed by the registered backend verifier and may leave only if

$$r_{t,k} = \max\left\{ \|\pi_{t,k}\mathbf{1} - a_t\|_1, \|\pi^\top_{t,k}\mathbf{1} - b_t\|_1 \right\} \leq \tau. \tag{4}$$

If verification fails, the lane remains active and can be reconsidered at a later audit. Thus a false-positive screen costs a verifier call, while a false-negative screen delays draining; neither can release an invalid result.

For every verified lane, DrainSinkhorn writes the output and compacts every candidate-indexed object: source and target descriptors when candidate specific, marginals, dual potentials, centered shifts, log weights, and scratch buffers. The next packed update sees only the surviving lanes. The logical-mask control uses the same update, screen, verifier, tolerance, and per-lane release times. It

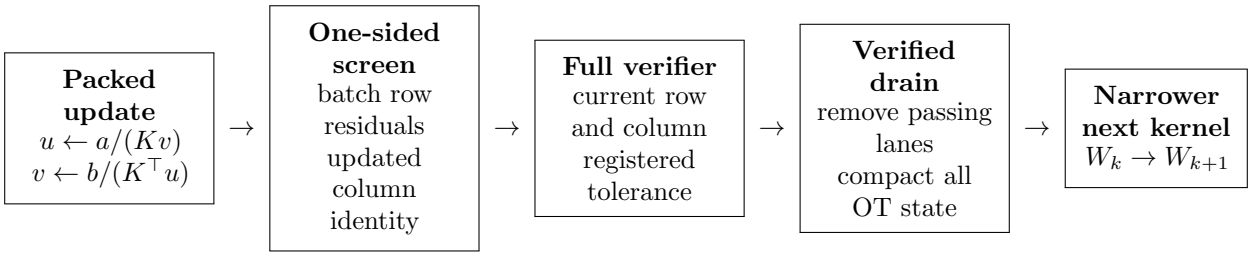

Figure 1: The numerical execution loop. A screen-negative lane continues. A screen-positive lane enters the current two-sided verifier. Passing lanes are physically removed from the batch; transport mass remains unchanged.

freezes a verified lane's potentials but retains the lane in the physical candidate tensor, so subsequent kernels still execute at the original width. Physical compaction instead removes the lane from every candidate-indexed tensor. Logical masking versus physical compaction therefore isolates the work eliminated by draining.

3.3 Execution modes and fallback

The implementation exposes four experimental modes. Width-one executes the project backend one problem at a time. Fixed-width packing updates every lane until a common final boundary. Logical masking records per-lane completion and freezes completed states without narrowing the kernel. Physical compaction adds verified draining. A frozen workload profile selects the deployable route. Width one is preferable when packing overhead exceeds the available reuse, while fixed-width execution can be preferable when depths are nearly uniform. A lane that reaches a minimum profitable width, a numerical anomaly, or an iteration limit falls back to the matched strong sequential path.

When a complete problem fits on one accelerator, different windows are sent to independent complete-device workers. This parallelizes problems without a collective in every Sinkhorn update. Row sharding supplies the capacity path for a single oversized problem. The main scale-out path assigns complete problems to independent devices.

4 From Release Depths to Eliminated Work

Let ℓ_t be the first production audit at which candidate t passes the current verifier, and let

$$A_k = \big|\{t : \ell_t \geq k\}\big| \tag{5}$$

be the surviving width before packed update k. Ignoring a separately recorded sequential tail, the number of executed candidate-update slots is the exact accounting identity

$$S_{\mathrm{drain}} = \sum_{k=1}^{\ell_{\max}} A_k, \qquad S_{\mathrm{mask}} = S_{\mathrm{static}} = W\ell_{\max}. \tag{6}$$

The area between the static rectangle and the survival curve is the work that draining removes. Logical masking changes which states are live but not the physical width seen by the kernel, so it retains the static slot count. The identity uses production detection depths ℓ_t measured on the actual audit grid.

Mapping slots to wall time also requires the width-dependent hardware cost. Let $c_k(A_k, \xi_k)$ be the measured cost of update k, where ξ_k records support shape, data type, and backend state. Then

$$\begin{aligned} T_{\mathrm{drain}} = \sum_k c_k(A_k, \xi_k) + H_{\mathrm{screen}} + H_{\mathrm{verify}} \\ + H_{\mathrm{compact}} + H_{\mathrm{route}}. \end{aligned} \tag{7}$$

Static execution replaces A_k with W and removes compaction. The resulting break-even condition is

$$\begin{aligned} \sum_k [c_k(W, \xi_k) - c_k(A_k, \xi_k)] > H_{\mathrm{screen}} + H_{\mathrm{verify}} \\ + H_{\mathrm{compact}} + H_{\mathrm{route}}. \end{aligned} \tag{8}$$

Equation (6) is exact post-run work accounting; Equation (7) becomes predictive only after measuring or modeling the conditional width-cost curve.

4.1 Why release depths differ

Depths vary with regularization, kernel geometry, marginals, proposal defect, transient dynamics, and local contraction. Nonlinear PF geometry supplies a local mechanism for the strict tail. After entering a contraction tube, a candidate residual can be represented schematically as

$$\rho_{t,k_0+s} \approx \sum_{j \geq 2} A_{t,j} \lambda_{t,j}^s, \tag{9}$$

where the nontrivial quotient modes $\lambda_{t,j}$ set decay rates and the modal amplitudes $A_{t,j}$ encode proposal alignment. A slow spectral mode has little effect on a natural trajectory when its initial amplitude is small. The model motivates depth profiling and audit placement. Alternating scaling supplies Equation (3), and the current residual supplies the release decision.

Table 1: Packer19 two-host attribution on one frozen input and release contract. C80 isolates audit and screening; C80-LM isolates logical masking from physical compaction. Intervals use an equal-host paired bootstrap over 24 host–order–GPU timing units.

ID	Change	Context	Time (s)	Ratio	95% CI	Result
C80	Batched audit	fixed	4.403→4.100	1.073	[1.0727,1.0737]	24/24 wins
C80	One-sided screen	fixed	4.100→3.887	1.055	[1.0547,1.0556]	24/24 wins
C80-LM	Logical freeze	fixed→mask	3.888→3.887	1.000	[0.9998,1.0003]	neutral
C80-LM	Physical drain	mask→compact	3.887→3.176	1.224	[1.2240,1.2247]	24/24 wins
C80-LM	Static packing	width-one→fixed	3.526→3.888	0.907	[0.9068,0.9079]	width-one wins
C80-LM	Full deployment	public→compact	45.066→3.176	14.193	[14.1849,14.2008]	complete path

5 Experiments

All formal speedups are baseline time divided by candidate time. Matched component comparisons hold inputs, proposal, tolerance, packed update, verifier, and consumer fixed. Full-deployment comparisons against the public Flash API are reported separately because they also include execution-path and backend differences.

5.1 Which mechanism creates the gain?

Two Packer19 campaigns run on two physical hosts, each with four A100-SXM4-80GB GPUs. C80 compares eight audit, screen, and retirement paths; C80-LM adds the missing fixed-width, logical-mask, and physical-compaction chain. Three frozen method orders produce 24 paired host–order–GPU timing units; three warmup-excluded repeats are summarized within each unit. The orders do not resample the biological workload. Each matched contrast freezes its input, source commit, Flash tree, tolerance, and single-cell transition consumer.

The attribution separates three mechanisms. C80 shows that batched two-sided residual evaluation and the Sinkhorn-specific one-sided screen each reduce time. C80-LM then holds per-lane release decisions fixed: logical masking alone is neutral at 1.00007× with interval 0.99981–1.00033×, whereas physical draining gives 1.224× and removes 100 of 256 physical candidate slots. The complete path reaches 14.193× against public Flash serial. The width-one/fixed ratio of 0.907× shows that plain static packing is unprofitable on this workload; the gain appears only when verified completion narrows the physical kernel.

The row-screen static and active paths each contain 744 screen-negative audit-lane events. Shadow full-verifier replay finds 0 lanes that already satisfied the release contract, and the largest batched-screen versus verifier-row discrepancy is 1.86×10^{-7}. All project outputs pass the 10^{-3} marginal contract and the frozen consumer check. In C80-LM, draining reduces median energy from 880 to 688 J, while its compacted buffers increase peak allocated memory from 80.8 to 106.9 MiB.

Table 2: M3 grouping intervention on one fixed real task multiset. Timing repeats are summarized within each condition and are not treated as independent datasets.

Grouping	H_D	Padding	Active slots	Static/active
heterogeneous	0.236	0.236	0.802	1.217×
homogeneous	0.127	0.110	0.892	1.105×
random 1	0.254	0.293	0.776	1.258×
random 2	0.264	0.270	0.789	1.235×
random 3	0.260	0.279	0.764	1.271×

Table 3: Real consumer-complete endpoints. Matched controls isolate draining; official/proposed measures the complete deployment.

Workload	Matched contrast [95% CI]	Support	Slots	Energy	Official/proposed
MetroPT-3	1.745 [1.745,1.746]×	3 seeds	5696→3128	1.783× lower	4.072×
Packer19	LM/PC 1.224 [1.2240,1.2247]×	24/24 wins	256→156	880→688 J	14.193×

5.2 Does depth heterogeneity create removable work?

M3 uses the same 32 real MetroPT problems and the same total correction work in every condition; only their grouping into windows changes. The oracle homogeneous grouping minimizes within-window depth variation, while heterogeneous and random groupings increase it. As padding rises, the static/active speedup grows from 1.105× to 1.217× and 1.235–1.271× on the random layouts. This fixed-work intervention links the survival curve in equation (5) to eliminated slots and measured wall time.

5.3 Real endpoint and deployment value

On the C66 MetroPT-3 failure-3 predictive-maintenance endpoint, matched current static/active time is 1.745× with bootstrap interval 1.745–1.746×. Candidate slots fall from 5696 to 3128, and endpoint energy falls by 1.783×. The complete active path is 4.072× faster than public Flash serial. The C80/C80-LM Packer19 consumer independently gives the component and deployment results in table 1. These endpoints consume the verified coupling through frozen alarm and cell-transition logic and report the result at the consumer boundary.

The C63 ImageNet-32 experiment carries the complete active execution path into 50,000 updates of PCA500 feature-space OT-Flow-Matching training. Across three paired seeds, the active deployment is 2.786× faster than public Flash serial, and every pre-specified feature-space quality ratio passes at NFE 4/8/16. This carries the execution gain through a real training pipeline while preserving its registered NFE quality contract.

Finally, C67 assigns complete windows to independent GPU workers. Four workers achieve 3.965–3.979× fixed-per-worker throughput scaling on held-out MetroPT routes. The global two-host, eight-worker cells use the same complete-device contract, with no Sinkhorn collective between workers.

Table 4: C67 host-1 fixed-per-worker throughput scaling from one to four complete-device workers.

Held-out route	Arm	4-worker throughput scaling
failure_2	official	3.979×
failure_2	static	3.967×
failure_2	active	3.979×
failure_3	official	3.976×
failure_3	static	3.965×
failure_3	active	3.975×

6 Operating Regime and Composition

DrainSinkhorn applies to batched positive EOT problems that share a compatible packed representation and expose a current residual. Its benefit is governed by the release-depth survival curve and the backend's width-cost curve. The router selects width-one execution for a single problem, static execution for nearly uniform depths, and active draining when the survival curve exposes enough removable padding. Irregular shapes are bucketed by compatible packed representation.

The outer layer composes with target-preserving inner solvers. FlashSinkhorn supplies the fused dense update used by the main experiments; OTT-JAX provides a separate matrix-free path. Alternative initialization, acceleration, or second-order methods can reduce each lane's release depth before the same screen–verify–drain contract is applied.

Safety is defined against the registered current two-sided verifier. A lane leaves the batch only after that verifier passes. Non-finite states return to the guarded correction path. A separate near-threshold finite-precision campaign covers 1/2/4/8-GPU configurations and FP64 escalation, while the main Flash timing path uses its upstream plan-application verifier.

Physical draining trades work for state movement and buffers. C80-LM measures about 26.1 MiB of additional peak allocation on Packer19 even as wall time and energy fall. This cost enters the routing gate together with the measured survival and width-cost curves.

7 Conclusion

Fast Sinkhorn kernels reduce the price of an update. DrainSinkhorn removes updates that no longer belong to unfinished problems. It does so with a numerical execution loop specific to EOT: alternating scaling exposes a cheap one-sided screen, the current two-sided residual preserves the release contract, and physical compaction turns verified completion into a narrower next kernel. The depth-survival identity states exactly which candidate slots disappear; the measured width-cost curve determines whether those slots translate into wall-time savings.

The formal attribution separates batched audit, Sinkhorn-specific screening, draining, and backend composition. Real MetroPT-3 and Packer19 consumers show that the isolated draining effect persists at endpoint and energy boundaries; ImageNet-32 feature-space OT-FM carries the complete path through training; complete-device workers scale the same contract without splitting a problem. Together these results establish safe elimination as the workload-level counterpart to fast inner Sinkhorn kernels: balance the mass, verify the solve, and drain the finished work.

Reproducibility. The release artifact records source and input hashes, commands, software and GPU environments, raw outputs, correctness gates, paired analyses, and the identity of every public baseline.

References

[1] Jason Altschuler, Jonathan Niles-Weed, and Philippe Rigollet. Near-linear time approximation algorithms for optimal transport via Sinkhorn iteration. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL `https://papers.nips.cc/paper_files/paper/2017/hash/491442df5f88c6aa018e86dac21d3606-Abstract.html`.

[2] Brandon Amos, Giulia Luise, Samuel Cohen, and Ievgen Redko. Meta optimal transport. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 791–813, 2023. URL `https://proceedings.mlr.press/v202/amos23a.html`.

[3] Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transportation distances. In *Advances in Neural Information Processing Systems*, volume 26, pages 2292–2300, 2013. URL `https://arxiv.org/abs/1306.0895`.

[4] Marco Cuturi, Laetitia Meng-Papaxanthos, Yingtao Tian, Charlotte Bunne, Geoff Davis, and Olivier Teboul. Optimal transport tools (OTT): A JAX toolbox for all things wasserstein, 2022. URL `https://arxiv.org/abs/2201.12324`.

[5] Jean Feydy, Thibault Séjourné, François-Xavier Vialard, Shun ichi Amari, Alain Trouvé, and Gabriel Peyré. Interpolating between optimal transport and MMD using sinkhorn divergences. In *Proceedings of the 22nd International Conference on Artificial Intelligence and Statistics*, volume 89 of *Proceedings of Machine Learning Research*, pages 2681–2690, 2019. URL `https://proceedings.mlr.press/v89/feydy19a.html`.

[6] Rémi Flamary, Nicolas Courty, Alexandre Gramfort, Mokhtar Z. Alaya, Aurélie Boisbunon, Stanislas Chambon, Laetitia Chapel, Adrien Corenflos, Kilian Fatras, Nemo Fournier, Léo Gautheron, Nathalie

T. H. Gayraud, Hicham Janati, Alain Rakotomamonjy, Ievgen Redko, Antoine Rolet, Antony Schutz, Vivien Seguy, Danica J. Sutherland, Romain Tavenard, Alexander Tong, and Titouan Vayer. POT: Python optimal transport. *Journal of Machine Learning Research*, 22(78):1–8, 2021. URL https://www.jmlr.org/papers/v22/20-451.html.

[7] Ginkgo Project. Batched solvers: Overview. Ginkgo user guide, 2026. URL https://gko-project.org/user-guide/concepts/batched.html. Accessed 2026-08-28.

[8] Mete Kemertas, Amir massoud Farahmand, and Allan D. Jepson. A truncated newton method for optimal transport, 2025. URL https://arxiv.org/abs/2504.02067.

[9] Aram-Alexandre Pooladian, Heli Ben-Hamu, Carles Domingo-Enrich, Brandon Amos, Yaron Lipman, and Ricky T. Q. Chen. Multisample flow matching: Straightening flows with minibatch couplings. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 28100–28127, 2023. URL https://proceedings.mlr.press/v202/pooladian23a.html.

[10] Alexey Radul, Brian Patton, Dougal Maclaurin, Matthew D. Hoffman, and Rif A. Saurous. Automatically batching control-intensive programs for modern accelerators. In *Proceedings of Machine Learning and Systems*, volume 2, 2020. URL https://proceedings.mlsys.org/paper_files/paper/2020/hash/39945d578f616735572174bf5e8f155d-Abstract.html.

[11] Meyer Scetbon, Marco Cuturi, and Gabriel Peyré. Low-rank Sinkhorn factorization. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 9344–9354, 2021. URL https://proceedings.mlr.press/v139/scetbon21a.html.

[12] Alexis Thibault, Lénaïc Chizat, Charles Dossal, and Nicolas Papadakis. Overrelaxed Sinkhorn–Knopp algorithm for regularized optimal transport, 2017. URL https://arxiv.org/abs/1711.01851.

[13] James Thornton and Marco Cuturi. Rethinking initialization of the sinkhorn algorithm. In *Proceedings of the 26th International Conference on Artificial Intelligence and Statistics*, volume 206 of *Proceedings of Machine Learning Research*, pages 8682–8698, 2023. URL https://proceedings.mlr.press/v206/thornton23a.html.

[14] John Viljoen, Johanna Haffner, Masayoshi Tomizuka, and Negar Mehr. Scaling nonlinear optimization: Many problems one GPU, 2026. URL https://arxiv.org/abs/2606.26341.

[15] Felix X.-F. Ye, Xingjie Li, An Yu, Ming-Ching Chang, Linsong Chu, and Davis Wertheimer. FlashSinkhorn: IO-aware entropic optimal transport on GPU, 2026. URL https://arxiv.org/abs/2602.03067.

[16] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation*, pages 521–538, 2022. URL https://www.usenix.org/conference/osdi22/presentation/yu.

[17] Stephen Zhang, Alireza Mousavi-Hosseini, Michal Klein, and Marco Cuturi. On fitting flow models with large sinkhorn couplings, 2025. URL https://arxiv.org/abs/2506.05526.

A Endpoint Contracts and Additional Detail

Endpoint levels. E1 terminates at residual-verified coupling output or potentials. E2 includes the frozen downstream consumer. E3 includes training, generation, or task evaluation. The main tables identify the level of every result.

C66 release-fair controls. Each workload runs independent Official public Flash serial, legacy static/active, and current static/active arms. Within the registered current contrast, static and active use the same inputs, cold proposal, regularization, tolerance, audit grid, current two-sided verifier, packed backend, and frozen consumer; physical draining is the controlled difference. Three order-balanced plan seeds are reused across four GPU placements. GPU placements and timing repeats stabilize endpoint measurement; the plan seed remains the independent experimental unit.

B Deployment and Operating Boundaries

Complete-device and row-sharded execution. When one problem fits on one accelerator, complete-device workers process different windows and require no Sinkhorn collective. Row sharding supplies a capacity path for an oversized single problem. Complete-device execution is the throughput path used by the scale-out results.